

DOMAIN 1: REACTJS

--- EASY ---

1. What is React and how does it differ from a framework like Angular?
2. What is JSX? Can you write React without it?
3. What is the difference between functional and class components?
4. What are props? Are they read-only?
5. What is state in React and how is it different from props?
6. What is the Virtual DOM and how does React use it?
7. What is the purpose of the key prop in lists?
8. What is useState? Give a simple example.
9. What is useEffect and when does it execute?
10. What is conditional rendering? Show two ways to do it.
11. How do you handle events in React?
12. What is the difference between element and component in React?
13. What does React.Fragment do and why would you use it?
14. What is a controlled component?
15. What is an uncontrolled component?
16. How do you pass data from parent to child?
17. How do you pass data from child to parent?
18. What is prop drilling?
19. What is the StrictMode in React? What does it do?
20. What is the difference between createElement and JSX?
21. What are default props?
22. What is the purpose of defaultValue vs value in form inputs?
23. How do you render a list in React?
24. What is the difference between null and undefined rendering in JSX?
25. What does ReactDOM.render do?
26. What is the root element in a React application?
27. How do you add inline styles in React?
28. What is the difference between className and class in JSX?
29. Can you return multiple elements from a component? How?
30. What is the purpose of index.js in a React project?

--- MEDIUM ---

1. Explain the full component lifecycle using hooks (mount, update, unmount).
2. What is the difference between useEffect with no deps, empty deps, and with deps?
3. What is the Context API and when would you use it over Redux?
4. Explain useCallback — what problem does it solve?
5. Explain useMemo — when does it actually help performance?
6. What is React.memo and how is it different from useMemo?
7. What is the difference between useRef and useState?
8. How do you implement a debounced search input using hooks?
9. What is code splitting in React and how do you implement it?
10. What is React.lazy and Suspense?
11. What is the difference between useEffect and useLayoutEffect?
12. How does React handle form validation?
13. What is the reconciliation algorithm?
14. What is the difference between PureComponent and React.memo?
15. How do you share logic between components without duplicating code?
16. What are custom hooks? Write a useFetch custom hook.
17. What is the purpose of useReducer and when would you prefer it over useState?
18. How do you handle side effects that need cleanup in useEffect?
19. What happens when you set state inside useEffect without dependencies?

20. How does batching work in React 18 vs React 17?
21. What is the difference between a presentational and a container component?
22. How do you implement pagination in React?
23. What is render prop pattern? When is it useful?
24. How would you implement a modal in React using portals?
25. What is the significance of the `act()` function in React testing?
26. How do you prevent unnecessary re-renders in a React app?
27. What is the difference between client-side and server-side rendering in React context?
28. How does React Router work? What is the difference between `BrowserRouter` and `HashRouter`?
29. What are protected routes in React Router?
30. How do you implement global error handling in React?

--- HARD ---

1. Explain React Fiber architecture — why was it built over the old Stack reconciler?
2. What is concurrent rendering in React 18? How is it different from synchronous rendering?
3. Explain `useTransition` and `useDeferredValue` — what problems do they solve?
4. What is a stale closure in React and how do you fix it inside `useEffect`?
5. How do you implement undo/redo functionality in React?
6. Explain the difference between HOC, render props, and custom hooks for code reuse.
7. How would you build a virtualized list (like `react-window`) from scratch?
8. What are error boundaries? What can they NOT catch?
9. How does `forwardRef` work? When do you need `useImperativeHandle` with it?
10. How would you design a form library (like `Formik`) from scratch in React?
11. How does React batch state updates in event handlers vs async code?
12. What is React Server Components (RSC)? How do they differ from SSR?
13. How would you optimize a React tree with 500+ deeply nested components?
14. What is hydration in React? What causes hydration mismatch errors?
15. How would you implement infinite scroll with intersection observer in React?
16. Explain the internal workings of Context — what causes all consumers to re-render?
17. How would you implement an optimistic UI update pattern in React?
18. What is Zustand and how does its subscription model differ from Redux?
19. Describe the scheduling mechanism in React 18's concurrent mode.
20. How would you migrate a large class-based React app to functional hooks?
21. What is the difference between controlled and uncontrolled inputs in terms of performance?
22. How do you implement drag and drop in React without a library?
23. Explain React's synthetic event system — why does it exist?
24. How would you build a real-time collaborative component (like a shared counter) in React?
25. What is the difference between `useEffect` for data fetching vs React Query / SWR? Why prefer libraries?
26. How does React handle memory cleanup for event listeners added in `useEffect`?
27. What is prop comparison in `React.memo` and when does the default comparison fail?
28. How would you implement a theme switcher with Context that doesn't re-render unrelated components?
29. Explain the role of the scheduler package in React internals.
30. How would you implement accessibility (a11y) features in a React SPA?

--- SUPER HARD ---

1. A component re-renders 60 times per second in production. Describe your complete debugging and fix strategy.
2. Implement React's `useState` from scratch using closures.
3. Design a micro-frontend architecture using React. How do you handle shared state across apps?
4. How would you implement React Server Components from scratch — what is the wire protocol?
5. Your React app's bundle is 4MB and initial load takes 8 seconds. Give a step-by-step optimization plan.
6. Implement a basic version of React's reconciliation algorithm for a simple VDOM.
7. Design a real-time whiteboard app in React where 50 users draw simultaneously.
8. How would you implement time-travel debugging (like Redux DevTools) for a custom state manager?
9. Explain exactly how `useContext` avoids prop drilling but still causes re-renders — and how to fix it.

10. Design a plugin system for a React app where third parties can inject components at runtime.
11. How would you implement React Suspense for data fetching without any library?
12. Build a React state manager that supports selectors, subscriptions, and batched updates.
13. How does React's event delegation work internally in React 17 vs React 16?
14. Describe how you would implement code splitting at the route AND component level simultaneously.
15. How would you build a production-grade form wizard with validation, state persistence, and multi-step navigation?

--- TRAP QUESTIONS ---

1. [TRAP] `useEffect` with empty deps array never re-runs — is that always true? (Trap: React StrictMode runs it twice in development.)
2. [TRAP] You can call hooks inside loops — True or False? (Trap: False. Hooks must always be called in the same order.)
3. [TRAP] `setState` is synchronous in React — True or False? (Trap: It's asynchronous and batched.)
4. [TRAP] The Virtual DOM is always faster than directly updating the real DOM — True or False? (Trap: For simple updates, direct DOM can be faster; React's advantage is in complex diffing.)
5. [TRAP] `React.memo` prevents all unnecessary re-renders — True or False? (Trap: Only if props pass shallow equality; object/function props created inline always fail comparison.)
6. [TRAP] `useEffect` runs before the browser paints — True or False? (Trap: `useEffect` runs AFTER paint. `useLayoutEffect` runs before.)
7. [TRAP] Context re-renders only the consuming component — True or False? (Trap: ALL consumers re-render when context value changes.)
8. [TRAP] You should always use `useCallback` for functions passed as props — True or False? (Trap: `useCallback` itself has overhead; only useful if the child is memoized.)
9. [TRAP] `key` prop must be unique across the entire app — True or False? (Trap: Keys only need to be unique among siblings.)
10. [TRAP] Updating state always triggers a re-render — True or False? (Trap: If new state equals old state via `Object.is`, React bails out.)
11. [TRAP] Functional components always re-render when parent re-renders — True or False? (Trap: Not if wrapped in `React.memo` with passing comparison.)
12. [TRAP] You can use `async` directly in `useEffect` — True or False? (Trap: `useEffect` callback cannot be `async` directly; must create inner `async` function.)
13. [TRAP] React's reconciliation always starts from the root — True or False? (Trap: It starts from the component that triggered `setState`.)
14. [TRAP] `Refs` cause re-renders when their value changes — True or False? (Trap: Changing `ref.current` does NOT trigger a re-render.)
15. [TRAP] `defaultProps` work with functional components in React 18+ — True or False? (Trap: They are deprecated for function components; use default parameter values instead.)

DOMAIN 2: NODE.JS

--- EASY ---

1. What is Node.js? How is it different from JavaScript in the browser?
2. What is the V8 engine?
3. What is non-blocking I/O?
4. What is the difference between `require()` and `import`?
5. What is the event loop?
6. What is npm? What is yarn?
7. What is the difference between dependencies and devDependencies?
8. What is Express.js?
9. How do you create a simple HTTP server in Node.js?
10. What is middleware in Express?
11. What is the difference between `app.use()` and `app.get()`?
12. What does `res.json()` do?
13. What is `req.params` vs `req.query` vs `req.body`?
14. How do you read a file asynchronously in Node.js?
15. What is the difference between `fs.readFile` and `fs.readFileSync`?
16. What is `process.env`?
17. What is the purpose of `.env` files?
18. What is nodemon used for?
19. What is `package.json`?
20. What does npm install do?
21. What is the purpose of `node_modules`?
22. What is a callback function?
23. What is callback hell?
24. What is a Promise?
25. What is `async/await`?
26. How do you handle errors in `async/await`?
27. What is the difference between `setTimeout` and `setInterval`?
28. What is `process.nextTick()`?
29. What is the difference between CommonJS and ESM modules?
30. What is the `__dirname` variable in Node.js?

--- MEDIUM ---

1. Explain all phases of the Node.js event loop in order.
2. What is the difference between `process.nextTick()` and `setImmediate()`?
3. What is libuv and what role does it play in Node.js?
4. What is the Node.js thread pool? What operations use it?
5. How does Node.js handle CPU-intensive tasks?
6. What are Worker Threads? When should you use them?
7. What is the cluster module and why is it needed?
8. How does Node.js achieve concurrency despite being single-threaded?
9. What is the difference between `spawn`, `exec`, `fork`, and `execFile` in `child_process`?
10. How do you implement JWT authentication in Express?
11. What is the difference between app-level and router-level middleware?
12. How do you handle file uploads in Node.js/Express?
13. What is CORS and how do you configure it in Express?
14. How do you implement rate limiting in Express?
15. How do you structure a large Node.js/Express application?
16. What is the difference between error-handling middleware and regular middleware?
17. How do you handle unhandled promise rejections in Node.js?
18. What are Node.js streams? What are the 4 types?
19. What is the `pipe()` method in streams?

20. How do you implement a REST API with proper HTTP status codes?
21. What is Mongoose? What does it add over the MongoDB native driver?
22. What are Mongoose virtuals?
23. What is population in Mongoose?
24. How do you validate request data in Express?
25. What is the purpose of the helmet package in Express?
26. How do you implement pagination in a Node.js API?
27. What is the difference between 401 and 403 status codes?
28. How do you implement refresh token rotation in Node.js?
29. What is WebSocket and how do you implement it in Node.js?
30. How does Node.js handle multiple simultaneous connections?

--- HARD ---

1. How does the V8 garbage collector work? Explain generational GC.
2. What are memory leaks in Node.js? How do you detect them in production?
3. How do you profile a Node.js app for CPU bottlenecks?
4. What is backpressure in streams and how do you handle it?
5. Explain the Transform stream — implement a custom one that uppercases text.
6. How does Node.js implement the observer pattern internally?
7. What is the EventEmitter? How does it differ from browser events?
8. Implement a rate limiter from scratch using the sliding window algorithm.
9. Implement a simple in-memory job queue in Node.js.
10. How do you achieve zero-downtime deployments with a Node.js server?
11. What is the difference between HTTP/1.1, HTTP/2, and HTTP/3 in Node.js?
12. How do you implement HTTP/2 server push in Node.js?
13. How do you implement a pub/sub system without a message broker in Node.js?
14. What are async iterators and async generators in Node.js?
15. How would you implement a distributed lock in Node.js without Redis?
16. What is the N-API and when would you write a native addon?
17. How do you secure a Node.js API against common OWASP vulnerabilities?
18. What is SSRF and how do you prevent it in a Node.js service?
19. How would you implement request tracing across microservices in Node.js?
20. Implement a connection pool from scratch in Node.js.
21. How does Node.js handle DNS lookups and why can they block the event loop?
22. What is graceful shutdown in Node.js and how do you implement it?
23. How would you implement a circuit breaker pattern in Node.js?
24. How do you implement server-sent events (SSE) in Node.js?
25. What is the difference between readable, writable, duplex, and transform streams?
26. How do you implement caching at the Node.js application layer without Redis?
27. How would you design a multi-tenant API in Node.js?
28. What is CQRS and how would you implement it in a Node.js service?
29. How do you handle database connection failures gracefully in a Node.js app?
30. Explain how to implement OAuth2 flow from scratch in Node.js.

--- SUPER HARD ---

1. Your Node.js service is handling 100k req/min and CPU is at 100%. Walk through your entire diagnosis and fix.
2. Design a real-time auction platform in Node.js handling 50,000 concurrent bidders.
3. Implement a distributed task scheduler in Node.js that guarantees exactly-once execution.
4. How would you implement a custom HTTP/2 multiplexer in Node.js?
5. Design a Node.js-based API gateway that handles auth, rate limiting, routing, and observability.
6. Implement a streaming ETL pipeline in Node.js processing 1M records/min with backpressure.
7. How would you implement a fully resilient microservice in Node.js with retry, circuit breaker, bulkhead, and timeout?
8. Build a Node.js event store (for event sourcing) with replay capability.
9. How does Node.js's require() caching work and how would you implement hot module reloading?

10. Design a WebSocket server in Node.js that scales across 10 nodes with message broadcasting.

--- TRAP QUESTIONS ---

1. [TRAP] Node.js is single-threaded so it cannot do parallel I/O — True or False? (Trap: libuv thread pool handles parallel I/O; only JS execution is single-threaded.)
2. [TRAP] `async/await` is non-blocking — True or False? (Trap: `await` itself yields but the awaited synchronous code inside the `async` function IS blocking.)
3. [TRAP] Adding more CPU cores always speeds up a Node.js app — True or False? (Trap: Only if you use `cluster` or `Worker Threads`; single-threaded apps use only 1 core.)
4. [TRAP] `process.nextTick()` runs after I/O callbacks — True or False? (Trap: It runs BEFORE I/O callbacks, at the end of the current operation.)
5. [TRAP] `Promise.all()` stops all promises when one fails — True or False? (Trap: It rejects immediately but other promises continue running; they just aren't awaited.)
6. [TRAP] `res.json()` automatically ends the response — True or False? (Trap: Yes, but calling `res.send()` after it throws 'headers already sent' — a common bug.)
7. [TRAP] Middleware is always executed in order — True or False? (Trap: Only if you call `next()`; missing `next()` silently hangs the request.)
8. [TRAP] `require()` is synchronous — True or False? (Trap: True — it blocks the event loop; this is why dynamic imports are preferred for large modules.)
9. [TRAP] Node.js can't handle file streaming — True or False? (Trap: Node.js streams are one of its strongest features for large file handling.)
10. [TRAP] Error-handling middleware is just regular middleware with a different signature — True or False? (Trap: It MUST have exactly 4 params (`err`, `req`, `res`, `next`) or Express won't treat it as error middleware.)

DOMAIN 3: MERN STACK

--- EASY ---

1. What does MERN stand for?
2. What is the role of MongoDB in the MERN stack?
3. What is the role of Express in the MERN stack?
4. What is the role of React in the MERN stack?
5. What is the role of Node.js in the MERN stack?
6. How does data flow from MongoDB to the React UI?
7. What is CORS and why does it matter in MERN?
8. What is the difference between SQL and MongoDB?
9. What is Mongoose?
10. What is a Mongoose Schema?
11. What is a Mongoose Model?
12. How do you connect Express to MongoDB using Mongoose?
13. What is axios and how do you use it to call an API from React?
14. What are the standard HTTP methods in REST?
15. What HTTP status codes represent success, client error, and server error?
16. What is a .env file and why should it never be committed to Git?
17. What is dotenv package used for?
18. What is JSON Web Token (JWT)?
19. How does JWT authentication work at a high level?
20. What is bcrypt used for in a MERN app?
21. What is the difference between authentication and authorization?
22. What is the purpose of the proxy field in React's package.json?
23. What is concurrently used for in MERN development?
24. What is the difference between ObjectId and a regular string ID in MongoDB?
25. What is a MongoDB collection vs a document?
26. What does populate() do in Mongoose?
27. What is the difference between find() and findOne() in Mongoose?
28. How do you delete a document in Mongoose?
29. What is the purpose of express.json() middleware?
30. What is the difference between req.params, req.query, and req.body?

--- MEDIUM ---

1. How would you implement JWT with refresh token rotation in a MERN app?
2. What is the difference between embedding and referencing in MongoDB schema design?
3. When would you embed vs reference in MongoDB?
4. What is the aggregation pipeline in MongoDB?
5. Write a MongoDB aggregation to get the top 5 users by total order value.
6. How do you implement pagination in a MERN API?
7. What is cursor-based pagination vs offset-based pagination?
8. How do you handle file uploads (images) in a MERN app?
9. What is Multer?
10. How do you store uploaded images — local disk vs cloud (Cloudinary/S3)?
11. What is Redux and when is it overkill?
12. What is Redux Toolkit (RTK)?
13. What is RTK Query and how does it compare to React Query?
14. How do you prevent NoSQL injection in MongoDB?
15. What is MongoDB indexing? What types of indexes exist?
16. When would you use a compound index in MongoDB?
17. How do you implement real-time features (notifications/chat) in MERN?
18. What is Socket.IO and how does it work?
19. How do you implement role-based access control (RBAC) in a MERN API?

20. How do you protect React routes based on user role?
21. What is server-side rendering (SSR) in the context of MERN? When would you use Next.js?
22. How do you handle errors globally in a MERN app (both frontend and backend)?
23. What is the difference between optimistic and pessimistic updates in React?
24. How do you manage loading and error states for API calls in React?
25. How do you implement search functionality in a MERN app?
26. What is MongoDB text search and when would you use it vs Elasticsearch?
27. How do you implement an email verification flow in MERN?
28. How do you implement forgot password / reset password in MERN?
29. What is CSRF and how do you prevent it in a MERN app?
30. What is XSS and how do you prevent it in a MERN React app?

--- HARD ---

1. Design a scalable MERN e-commerce platform for 1M users. What architecture decisions would you make?
2. What is MongoDB Change Streams? How would you use it for real-time UI updates?
3. How do you implement multi-tenancy in a MERN application?
4. What is the CAP theorem and where does MongoDB fit?
5. How do you handle distributed transactions in MongoDB (multi-document)?
6. What are MongoDB transactions and how do you use them with Mongoose?
7. What is sharding in MongoDB? When do you need it?
8. What is replication in MongoDB? How does failover work?
9. How would you implement an event-driven architecture in a MERN stack?
10. How would you implement an audit log system in a MERN app?
11. How do you implement database migrations in MongoDB for a live production system?
12. What is the aggregation \$lookup stage? Write a complex aggregation with multiple lookups.
13. How would you design a newsfeed (like Instagram) in MERN — push vs pull model?
14. How would you implement full-text search across multiple collections in MongoDB?
15. How do you implement soft delete in MongoDB and ensure it doesn't affect queries?
16. How would you design a MERN app that works offline-first with sync on reconnect?
17. How do you handle race conditions in MongoDB without transactions?
18. What is an upsert operation in MongoDB and when is it useful?
19. How would you implement a recommendation engine in a MERN app?
20. How do you implement webhooks in a MERN backend?
21. What is GraphQL and how would it change the architecture of a MERN app?
22. How do you implement a MERN app with microservices — what changes vs monolith?
23. How would you implement server-side caching in a MERN API?
24. How do you test a MERN application (unit, integration, e2e)?
25. How would you implement a complete CI/CD pipeline for a MERN app?
26. What are MongoDB Atlas triggers and how would you use them?
27. How do you implement data encryption at rest and in transit in MERN?
28. How would you handle session management across multiple Node.js instances?
29. What is the mongoose-paginate-v2 plugin and how does it work under the hood?
30. How do you implement a MERN app with WebRTC for video calling?

--- SUPER HARD ---

1. Your MERN app gets 100k requests/min and MongoDB becomes the bottleneck. Give a step-by-step scaling plan.
2. Design a MERN-based multi-tenant SaaS with separate databases per tenant and shared auth.
3. How would you implement CQRS and Event Sourcing in a MERN stack?
4. Your React frontend, Node backend, and MongoDB are all on different infrastructure. Describe all the security boundaries you would set up.
5. Design a complete observability stack (logs, metrics, traces) for a MERN application in production.
6. How would you implement a distributed cache invalidation strategy in a MERN app?
7. Design a MERN app where 10,000 users can collaborate on the same document simultaneously.
8. How would you migrate a 500GB MongoDB database with zero downtime?

9. Build a MERN-based data pipeline that ingests, transforms, and stores 1M records/day.
10. How would you implement a MERN app that passes SOC2 compliance requirements?

--- TRAP QUESTIONS ---

1. [TRAP] MongoDB is schema-less so you don't need to think about data design — True or False? (Trap: Poor schema design is the #1 cause of MongoDB performance issues.)
2. [TRAP] JWT stored in localStorage is secure — True or False? (Trap: Vulnerable to XSS. HttpOnly cookies are safer.)
3. [TRAP] mongoose.connect() needs to be called before every query — True or False? (Trap: Connect once at startup; Mongoose queues queries until connected.)
4. [TRAP] You should always use ObjectId for relationships in MongoDB — True or False? (Trap: Sometimes string IDs or custom IDs are better for readability and cross-system compatibility.)
5. [TRAP] Express automatically handles async errors — True or False? (Trap: You must explicitly call next(err) or wrap in try-catch.)
6. [TRAP] React's fetch/axios calls should go directly in the component body — True or False? (Trap: They must go inside useEffect to avoid infinite loops.)
7. [TRAP] MongoDB find() returns an array — True or False? (Trap: It returns a Cursor; you must call .toArray() or iterate it.)
8. [TRAP] CORS only affects POST requests — True or False? (Trap: CORS preflight applies to any cross-origin request that isn't a simple request.)
9. [TRAP] bcrypt.compare() is case-sensitive — True or False? (Trap: Yes, passwords ARE case-sensitive. And bcrypt includes the salt inside the hash automatically.)
10. [TRAP] JWT expiry is enforced by the server — True or False? (Trap: JWT is stateless; expiry is checked by the verifier but the token itself isn't revoked without a blacklist.)

DOMAIN 4: AI / ML

--- EASY ---

1. What is the difference between AI, ML, and Deep Learning?
2. What is supervised learning? Give 3 examples.
3. What is unsupervised learning? Give 3 examples.
4. What is the difference between classification and regression?
5. What is a training set, validation set, and test set?
6. What is overfitting?
7. What is underfitting?
8. What is a confusion matrix?
9. What is precision and recall?
10. What is the F1 score?
11. What is accuracy? When is it a bad metric?
12. What is gradient descent?
13. What is a loss function?
14. What is the bias-variance tradeoff?
15. What is a feature?
16. What is feature engineering?
17. What is normalization vs standardization?
18. What is a train-test split?
19. What is cross-validation?
20. What is k-fold cross-validation?
21. What is linear regression?
22. What is logistic regression?
23. What is a decision tree?
24. What is a random forest?
25. What is a neural network at a high level?
26. What is an activation function? Name 3.
27. What is the difference between a parameter and a hyperparameter?
28. What is data augmentation?
29. What is transfer learning?
30. What is inference vs training in ML?

--- MEDIUM ---

1. Explain backpropagation step by step.
2. What is the vanishing gradient problem? How do ReLU and batch normalization help?
3. What is the exploding gradient problem? How do you fix it?
4. What is dropout regularization? When should you NOT use it?
5. What is L1 vs L2 regularization? What effect does each have on weights?
6. What is the difference between bagging and boosting?
7. What is XGBoost and why is it so powerful on tabular data?
8. Explain the curse of dimensionality.
9. What is PCA (Principal Component Analysis)?
10. What is the difference between generative and discriminative models?
11. What is k-means clustering? What are its limitations?
12. What is DBSCAN clustering and when is it better than k-means?
13. What is the elbow method for choosing k in k-means?
14. What is an ROC curve and AUC score?
15. What does AUC = 0.5 mean vs AUC = 1.0?
16. What is the difference between micro-average and macro-average in multi-class metrics?
17. What is a CNN? What do convolutional, pooling, and fully connected layers do?
18. What is the receptive field in a CNN?
19. What is max pooling vs average pooling?

20. What is batch normalization and where do you place it?
21. What is an RNN and what problem does it solve?
22. What is LSTM and what problem does it solve over vanilla RNN?
23. What is the difference between LSTM and GRU?
24. What is word embedding? What is Word2Vec?
25. What is the difference between CBOW and Skip-gram in Word2Vec?
26. What is semantic similarity in NLP?
27. How do you handle imbalanced datasets?
28. What is SMOTE?
29. What is early stopping in training?
30. What is learning rate scheduling?

--- HARD ---

1. Explain the attention mechanism in transformers from scratch.
2. What is multi-head attention and why is it better than single-head?
3. What is positional encoding in transformers and why is it needed?
4. What is BERT? How does masked language modeling work?
5. What is GPT? How does causal/autoregressive language modeling work?
6. What is the difference between encoder-only, decoder-only, and encoder-decoder models?
7. What is GAN? Explain the training dynamic between generator and discriminator.
8. What is mode collapse in GANs and how do you address it?
9. What is a VAE (Variational Autoencoder)?
10. What is the reparameterization trick in VAEs?
11. How do you handle class imbalance in a CNN trained on medical images?
12. Design a full ML pipeline for crop disease detection — from data collection to deployment.
13. What is data leakage in ML? Give 3 examples of how it occurs.
14. What is concept drift and how do you monitor for it in production?
15. What is the difference between online learning and batch learning?
16. How do you implement model versioning in a production ML system?
17. What is A/B testing in the context of ML models?
18. What is shadow deployment of ML models?
19. What is feature importance? How does it differ between tree models and neural networks?
20. What is SHAP and how does it explain model predictions?
21. What is LIME and how does it differ from SHAP?
22. What is model calibration? Why does a high-accuracy model have poor calibration?
23. Explain the difference between frequentist and Bayesian approaches to ML.
24. What is a Gaussian process and when is it used?
25. What is hyperparameter tuning? Compare grid search, random search, and Bayesian optimization.
26. What is the difference between mean squared error and mean absolute error as loss functions?
27. What is Huber loss?
28. How would you implement a real-time inference pipeline with < 100ms latency?
29. What is model distillation (knowledge distillation)?
30. What is quantization in the context of ML model deployment?

--- SUPER HARD ---

1. Your CNN has 97% train accuracy and 62% test accuracy on a crop disease dataset. Walk me through every possible cause and fix.
2. Design a federated learning system for training a medical imaging model without centralizing patient data.
3. Implement the attention mechanism in NumPy from scratch.
4. How would you design a multi-modal AI system (image + text + tabular) for agricultural advisory?
5. Explain RLHF (Reinforcement Learning from Human Feedback) end to end — from SFT to PPO fine-tuning.
6. How does diffusion model work mathematically? Explain the forward and reverse process.
7. Design an ML monitoring system that detects data drift, model drift, and prediction drift in real time.
8. How would you train a model on 100 TB of image data across 1000 GPUs?

9. Implement a custom training loop in PyTorch with gradient accumulation, mixed precision, and distributed training.
10. How do you build an ML system that is fair and unbiased for different demographic groups?
11. What is Neural Architecture Search (NAS) and how does it work?
12. How would you implement continual learning to prevent catastrophic forgetting?
13. Design a self-supervised pretraining pipeline for domain-specific image data with no labels.
14. Explain the math behind the Transformer's scaled dot-product attention.
15. How would you reduce a 1B parameter model to run on a mobile device with <100MB memory?

--- TRAP QUESTIONS ---

1. [TRAP] Higher accuracy always means a better model — True or False? (Trap: On 95% negative-class datasets, always predicting negative gives 95% accuracy but is useless.)
2. [TRAP] More training data always improves performance — True or False? (Trap: Noisy or mislabeled data hurts performance.)
3. [TRAP] Deep learning always beats classical ML — True or False? (Trap: On small/tabular data, XGBoost routinely outperforms deep learning.)
4. [TRAP] If train and test accuracy are both high, the model is good — True or False? (Trap: Data leakage can cause this illusion.)
5. [TRAP] A model with low loss is always production-ready — True or False? (Trap: Loss on validation set doesn't reflect real-world distribution shift.)
6. [TRAP] Normalization is always required before training — True or False? (Trap: Tree-based models like Random Forest don't require normalization.)
7. [TRAP] ReLU solves the vanishing gradient problem completely — True or False? (Trap: ReLU has the dying ReLU problem — neurons can become permanently inactive.)
8. [TRAP] Cross-validation prevents overfitting — True or False? (Trap: It detects overfitting but doesn't prevent it; you still need regularization.)
9. [TRAP] A confusion matrix is only for binary classification — True or False? (Trap: It works for multi-class as well, as an NxN matrix.)
10. [TRAP] Larger batch size always leads to better generalization — True or False? (Trap: Larger batches often generalize worse; small batches add noise that acts as regularization.)
11. [TRAP] Dropout should be applied during inference — True or False? (Trap: Dropout is disabled during inference; only active during training.)
12. [TRAP] Transfer learning always improves performance — True or False? (Trap: If the source domain is very different from the target domain, it can hurt.)

DOMAIN 5: GENERATIVE AI

--- EASY ---

1. What is a Large Language Model (LLM)?
2. What is a prompt in the context of LLMs?
3. What is prompt engineering?
4. What is the difference between zero-shot, one-shot, and few-shot prompting?
5. What is tokenization? How do LLMs count tokens?
6. What does 'temperature' control in LLM generation?
7. What is top-p (nucleus) sampling?
8. What is the difference between temperature=0 and temperature=1?
9. What is hallucination in LLMs?
10. What is a context window?
11. What is RAG (Retrieval Augmented Generation)?
12. What is a vector database?
13. What is an embedding?
14. What is cosine similarity?
15. What is LangChain?
16. What is a chain in LangChain?
17. What is a prompt template?
18. What is Groq? Why is it fast?
19. What is Ollama used for?
20. Name 5 popular LLMs available via API.
21. What is fine-tuning an LLM?
22. What is system prompt vs user prompt?
23. What is max_tokens in an LLM API call?
24. What is streaming in the context of LLM responses?
25. What is the difference between GPT-3 and GPT-4 at a conceptual level?
26. What are AI agents?
27. What is the ReAct (Reason + Act) pattern in AI agents?
28. What is tool use / function calling in LLMs?
29. What is a system message?
30. What is a chat completion API?

--- MEDIUM ---

1. Explain the transformer architecture — encoder, decoder, attention.
2. What is self-attention? How is it computed?
3. What is the difference between encoder-only (BERT) and decoder-only (GPT) models?
4. What is the difference between fine-tuning and RAG? When do you choose each?
5. What is chunking in RAG? What are the tradeoffs of chunk size?
6. What chunking strategies exist — fixed size, sentence, semantic, recursive?
7. What is a retriever in RAG? What are dense vs sparse retrievers?
8. What is BM25? How does it differ from semantic search?
9. What is hybrid search in RAG?
10. What is a reranker and why do you use one after retrieval?
11. What is the difference between FAISS and ChromaDB?
12. What is LangGraph? What problem does it solve over LangChain?
13. What is chain-of-thought (CoT) prompting?
14. What is self-consistency prompting?
15. What is tree-of-thought prompting?
16. What is LLM-as-Judge?
17. What is RAGAS and what metrics does it use to evaluate RAG?
18. What metrics would you use to evaluate a RAG pipeline?
19. What is faithfulness in RAG evaluation?

20. What is answer relevance in RAG evaluation?
21. What is context precision and context recall in RAG?
22. What is a memory module in an AI agent?
23. What is the difference between short-term and long-term memory in agents?
24. What is multi-agent systems?
25. How does LLM function calling work technically?
26. What is structured output in LLMs?
27. What is Pydantic and how is it used with LLM outputs?
28. What is LCEL (LangChain Expression Language)?
29. What is an LLM router chain?
30. What is the difference between synchronous and async LLM API calls?

--- HARD ---

1. Design a production-grade RAG pipeline with ingestion, chunking, embedding, retrieval, reranking, generation, and evaluation.
2. What is Corrective RAG (CRAG) and how does it improve naive RAG?
3. What is Self-RAG and how does it differ from CRAG?
4. How do you implement a LangGraph Corrective RAG pipeline with a loop?
5. What is prompt injection and how do you defend against it in production?
6. How do you implement streaming LLM responses in a full-stack web app?
7. What is PEFT (Parameter-Efficient Fine-Tuning)? Name 3 methods.
8. What is LoRA (Low-Rank Adaptation)? How does it reduce trainable parameters?
9. What is QLoRA and how does it differ from LoRA?
10. How would you fine-tune an LLM for a domain-specific task with limited data?
11. How do you reduce LLM API costs in a high-traffic application?
12. What is caching in the context of LLM applications? What is semantic caching?
13. How do you implement multi-turn conversation memory in a RAG system?
14. What is the lost-in-the-middle problem in LLMs with long contexts?
15. How does context window size affect retrieval strategy in RAG?
16. What is a knowledge graph and how would you combine it with RAG?
17. How do you handle multi-lingual queries in a RAG system?
18. What is ColBERT and how does it differ from bi-encoder retrieval?
19. How do you evaluate hallucination rate in an LLM application?
20. What is Constitutional AI and how does it work?
21. What is DPO (Direct Preference Optimization) vs PPO for RLHF?
22. How would you implement a ReAct agent from scratch without LangChain?
23. What is Mixture of Experts (MoE) architecture?
24. How does speculative decoding speed up LLM inference?
25. What is FlashAttention and how does it optimize transformer inference?
26. How do you implement a guardrails system for an LLM application?
27. What is an LLM gateway and what capabilities should it have?
28. How do you implement observability (tracing, logging) for an LLM application?
29. What is LangSmith and how does it help with LLM debugging?
30. How would you design a multi-agent debate system where agents challenge each other?

--- SUPER HARD ---

1. Design a complete AI agricultural assistant (like Kissan AI) end-to-end: voice input, image analysis, RAG for crop knowledge, weather context, and LLM response.
2. Your RAG pipeline has 70% faithfulness and 55% answer relevance. Walk me through diagnosing and improving each metric.
3. Design a multi-agent system where a planner, researcher, coder, and critic agent collaborate on a task.
4. How would you implement a production LLM evaluation framework from scratch with automated test cases?
5. Design an LLM-powered code review system that evaluates correctness, security, performance, and style.
6. How would you build a real-time LLM application that handles 10,000 concurrent users with streaming responses?

7. Explain the internals of how a token is generated — from logits to sampling to detokenization.
8. Design a self-improving RAG system that uses user feedback to improve retrieval and generation over time.
9. How would you implement a secure, multi-tenant LLM API gateway where each tenant has isolated context and rate limits?
10. How would you measure and reduce the carbon footprint of training and serving an LLM?

--- TRAP QUESTIONS ---

1. [TRAP] Larger context window = better model — True or False? (Trap: Lost-in-the-middle problem — attention degrades for content in the middle of long contexts.)
2. [TRAP] RAG eliminates hallucination — True or False? (Trap: RAG reduces it; the LLM can still hallucinate or misinterpret retrieved context.)
3. [TRAP] Temperature = 0 always gives the same output — True or False? (Trap: Not guaranteed across all models and frameworks due to floating point non-determinism.)
4. [TRAP] More retrieved chunks = better answers — True or False? (Trap: Too many chunks add noise and can confuse the LLM.)
5. [TRAP] Embeddings are just word counts — True or False? (Trap: Embeddings are dense semantic vectors learned by neural networks.)
6. [TRAP] Fine-tuning is always better than RAG for domain knowledge — True or False? (Trap: Fine-tuning doesn't add new knowledge reliably; RAG is better for factual grounding.)
7. [TRAP] System prompt is always private and secure from users — True or False? (Trap: System prompts can be leaked via prompt injection attacks.)
8. [TRAP] LLMs understand text the way humans do — True or False? (Trap: LLMs predict tokens based on statistical patterns, not semantic understanding.)
9. [TRAP] Chunk size doesn't matter much for RAG — True or False? (Trap: Chunk size critically affects retrieval precision and context completeness.)
10. [TRAP] An agent with more tools is always smarter — True or False? (Trap: Too many tools confuse the LLM and increase hallucinated tool calls.)

DOMAIN 6: PYTHON DEVELOPER

--- EASY ---

1. What is Python? What are its key features?
2. What is the difference between a list, tuple, and set?
3. What is a dictionary in Python?
4. What are Python decorators?
5. What is a generator?
6. What is the difference between *args and **kwargs?
7. What is the difference between is and ==?
8. What is list comprehension?
9. What is dictionary comprehension?
10. What is a lambda function?
11. What is the difference between append() and extend()?
12. What is the GIL (Global Interpreter Lock)?
13. What is the difference between deepcopy and copy?
14. What is the difference between @staticmethod and @classmethod?
15. What is the difference between __str__ and __repr__?
16. What is the with statement and what is a context manager?
17. What is exception handling in Python? Name 5 built-in exceptions.
18. What is the difference between range() and xrange()? (Python 2 vs 3)
19. What is PEP 8?
20. What is None in Python?
21. What is the difference between mutable and immutable types?
22. What are Python's built-in data types?
23. What is type hinting in Python?
24. What is the zip() function?
25. What is map() and filter()?
26. What is enumerate()?
27. What is the difference between sorted() and .sort()?
28. What is a set comprehension?
29. What is the walrus operator (:=)?
30. What is f-string formatting?

--- MEDIUM ---

1. Explain Python's MRO (Method Resolution Order) with multiple inheritance example.
2. What is Python's descriptor protocol (__get__, __set__, __delete__)?
3. What are metaclasses?
4. What are slots (__slots__) in Python classes?
5. What is the difference between threading, multiprocessing, and asyncio?
6. When would you use asyncio vs threads vs multiprocessing?
7. What is a coroutine in Python?
8. What is the difference between a coroutine and a generator?
9. What is the event loop in asyncio?
10. How does Python's garbage collector handle circular references?
11. What is functools.lru_cache? How does it work?
12. What is the difference between @property and a regular method?
13. What is Python's data model (__dunder__ methods)?
14. What is __new__ vs __init__?
15. What is an Abstract Base Class (ABC)?
16. What is duck typing?
17. What is the difference between ABC and Protocol (structural subtyping)?
18. How does Python implement operator overloading?
19. What is the itertools module used for?

20. What is `functools.partial`?
21. What is `functools.wraps` and why is it important for decorators?
22. How do you implement a decorator with arguments?
23. How do you implement a class-based decorator?
24. What is the `collections` module? Name 4 useful classes from it.
25. What is a `namedtuple`? When would you use it over a `dataclass`?
26. What is a `dataclass` in Python?
27. What is the difference between `dataclass` and `NamedTuple`?
28. What is the `pathlib` module?
29. How do you serialize/deserialize Python objects (`pickle`, `json`)?
30. What is the difference between `json.dumps` and `json.dump`?

--- HARD ---

1. Implement a thread-safe singleton in Python.
2. Implement memoization using a decorator that handles unhashable arguments.
3. How does Python's memory model handle integer caching (small ints)?
4. Explain Python's reference counting and when it fails.
5. How does Python's `asyncio` event loop differ from Node.js's?
6. Implement a custom context manager using both `__enter__`/`__exit__` and `contextlib`.
7. Implement a custom iterator and iterable from scratch.
8. What are Python's abstract syntax trees (AST) and how would you use them?
9. How does Python import work internally? What is `sys.modules`?
10. What is monkey patching? When is it acceptable?
11. How would you implement a plugin system in Python?
12. What is the difference between `__getattr__` and `__getattribute__`?
13. Implement a lazy property decorator.
14. How do you implement C extensions in Python?
15. What is Cython and when would you use it?
16. How do you profile Python code for memory usage?
17. How do you profile Python code for CPU performance?
18. What are Python's weak references and when are they useful?
19. How does Python's `sys.settrace` work? How is `pdb` implemented with it?
20. Implement a simple observer pattern (event system) in Python.
21. What is the difference between `queue.Queue` and `asyncio.Queue`?
22. Implement a thread pool from scratch in Python.
23. How would you implement a rate limiter as a Python decorator?
24. What is the difference between global interpreter lock and thread safety?
25. How do you write Python code that is both thread-safe and `async`-safe?
26. What is the `__init_subclass__` hook?
27. How do Python descriptors power `@property` and `@classmethod` internally?
28. What is a `__class_cell__` and when does it matter?
29. Implement a simple dependency injection container in Python.
30. How would you build a CLI tool with argument parsing, config files, and plugins?

--- SUPER HARD ---

1. Implement Python's `asyncio` event loop from scratch using selectors.
2. Implement a mini ORM (Object Relational Mapper) in Python.
3. How would you implement Python bytecode optimization for a hot code path?
4. Design a Python framework for building REST APIs from scratch (like Flask).
5. How would you implement a reactive programming library in Python?
6. How does CPython's frame object work? How does `eval.c` execute bytecode?
7. Implement a coroutine scheduler from scratch without `asyncio`.
8. Design a Python testing framework from scratch (like `pytest`).
9. How would you implement a garbage collector in Python from scratch?

10. Build a Python metaclass that enforces interface contracts at class creation time.

--- TRAP QUESTIONS ---

1. [TRAP] Mutable default arguments are fine — True or False? (Trap: `def f(x=[])` shares the list across ALL calls — classic Python gotcha.)
2. [TRAP] Python is always pass-by-reference — True or False? (Trap: Python is pass-by-object-reference; immutables feel like pass-by-value.)
3. [TRAP] GIL prevents all race conditions — True or False? (Trap: GIL only prevents simultaneous bytecode execution; data races in I/O or C extensions still occur.)
4. [TRAP] `del x` frees memory — True or False? (Trap: `del` removes the name binding; memory is freed only when reference count hits 0.)
5. [TRAP] `__eq__` override doesn't affect hashing — True or False? (Trap: If you override `__eq__`, Python sets `__hash__` to `None`, breaking dict/set usage.)
6. [TRAP] Generators are always more memory efficient — True or False? (Trap: Generators have overhead per yield; for small datasets, a list can be faster.)
7. [TRAP] `is None` and `== None` are equivalent — True or False? (Trap: Use `'is None'`; `__eq__` can be overridden to make `'== None'` behave unexpectedly.)
8. [TRAP] Python integers are fixed-size like in C — True or False? (Trap: Python ints are arbitrary precision; very large ints are slow to compute.)
9. [TRAP] Threading in Python achieves true parallelism — True or False? (Trap: Not for CPU-bound tasks due to GIL; use multiprocessing instead.)
10. [TRAP] Class variables are independent of instances — True or False? (Trap: If you mutate a class-level mutable (list/dict), all instances see the change.)

DOMAIN 7: DATA STRUCTURES & ALGORITHMS

--- EASY ---

1. What is Big-O notation?
2. What is the time complexity of binary search?
3. What is the difference between a stack and a queue?
4. What is a linked list? Types?
5. What is a binary tree?
6. What is a binary search tree (BST)?
7. What is BFS? What data structure does it use?
8. What is DFS? What are the two common implementations?
9. What is a hash table?
10. What is a collision in hashing? Name 2 collision resolution methods.
11. What is the difference between an array and a linked list?
12. What is a dynamic array?
13. What is the time complexity of inserting at the beginning of a linked list?
14. What is the time complexity of inserting at the end of an array?
15. What is a priority queue?
16. What is a heap?
17. What is the difference between min-heap and max-heap?
18. What is a graph? What are the two ways to represent a graph?
19. What is a directed vs undirected graph?
20. What is a weighted graph?
21. What is recursion? What is a base case?
22. What is factorial recursively?
23. What is Fibonacci recursively? What is its time complexity?
24. What is the difference between greedy and dynamic programming?
25. What is memoization?
26. What is tabulation?
27. What is sorting? Name 5 sorting algorithms.
28. What is the time complexity of bubble sort?
29. What is the time complexity of merge sort?
30. What is the time complexity of quicksort (best, average, worst)?

--- MEDIUM ---

1. Explain Dijkstra's algorithm — what data structure makes it efficient?
2. What is the time complexity of Dijkstra's with a min-heap?
3. What is Bellman-Ford and when does Dijkstra fail?
4. What is Floyd-Warshall algorithm?
5. What is topological sorting? Implement using Kahn's algorithm.
6. What is a trie? What is its time complexity for insert and search?
7. What is a segment tree? What operations does it support?
8. What is a Fenwick tree (Binary Indexed Tree)?
9. What is the two-pointer technique? Give 3 examples.
10. What is the sliding window technique? Give 3 examples.
11. What is dynamic programming? Solve the 0/1 knapsack problem.
12. What is the coin change problem? Solve it with DP.
13. What is longest common subsequence (LCS)? Solve it with DP.
14. What is longest increasing subsequence (LIS)?
15. What is the edit distance problem (Levenshtein distance)?
16. What is Floyd's cycle detection algorithm?
17. How do you find the middle of a linked list in one pass?
18. How do you reverse a linked list iteratively and recursively?
19. What is a doubly linked list vs singly linked list?

20. How do you detect a cycle in a directed graph?
21. How do you find connected components in a graph?
22. What is union-find (disjoint set union)?
23. What is path compression and union by rank in DSU?
24. What is Kruskal's algorithm for MST?
25. What is Prim's algorithm for MST?
26. What is the difference between DFS preorder, inorder, and postorder?
27. How do you find the lowest common ancestor (LCA) in a BST?
28. What is a balanced BST? Name 2 types.
29. What is an AVL tree?
30. What is the difference between insertion sort and selection sort?

--- HARD ---

1. Implement LRU Cache with $O(1)$ get and put.
2. Implement LFU Cache.
3. Find median of a data stream using two heaps.
4. Design a data structure that supports insert, delete, and getRandom in $O(1)$.
5. Implement a stack using two queues and vice versa.
6. Implement a min-stack that supports getMin() in $O(1)$.
7. Solve the trapping rain water problem — explain both $O(n)$ approaches.
8. Solve the largest rectangle in histogram problem.
9. Explain the KMP string matching algorithm — why is it $O(n+m)$?
10. Explain Rabin-Karp rolling hash algorithm.
11. What is Z-algorithm for string matching?
12. Solve the word search II problem using a Trie.
13. What is a Red-Black Tree? What are its balancing properties?
14. What is B-tree and B+ tree? How are they used in databases?
15. What is amortized analysis? Explain dynamic array resize cost.
16. What are skip lists? How do they achieve $O(\log n)$ operations?
17. What is the difference between NP, NP-Complete, and NP-Hard?
18. What is a polynomial-time reduction?
19. Solve the N-Queens problem using backtracking.
20. Solve the Sudoku solver problem using backtracking.
21. What is the Travelling Salesman Problem (TSP)? How do you solve it for small n ?
22. What is a suffix array and suffix tree?
23. Implement a consistent hash ring.
24. What is a van Emde Boas tree?
25. What is the difference between a Bloom filter and a hash set?
26. Solve maximum flow problem — explain Ford-Fulkerson.
27. What is a bipartite graph? How do you check if a graph is bipartite?
28. What is Tarjan's algorithm for strongly connected components?
29. Implement a persistent segment tree.
30. What is offline vs online algorithm design?

--- SUPER HARD ---

1. Design and implement a distributed hash table (DHT) like Chord.
2. Implement a concurrent lock-free hash map in C++ (describe the approach in detail).
3. Implement the RAFT consensus algorithm at a design level.
4. How would you solve 2-SAT (satisfiability) problem in $O(n+m)$?
5. Implement heavy-light decomposition on a tree.
6. What is centroid decomposition and when do you use it?
7. Implement a wavelet tree.
8. Solve the maximum weighted bipartite matching problem.
9. Implement the Knuth-Morris-Pratt and Aho-Corasick algorithms and compare their use cases.

10. Design a data structure supporting range updates and range queries in $O(\log n)$ with lazy propagation.
11. What is link-cut tree and when is it used over segment tree?
12. Implement a mergeable heap.
13. What is the best algorithm for matrix exponentiation? Solve Fibonacci in $O(\log n)$.
14. Implement a fully persistent balanced BST.
15. Solve the shortest path problem on a graph with negative weights and explain why Dijkstra fails.

--- TRAP QUESTIONS ---

1. [TRAP] HashMap lookup is always $O(1)$ — True or False? (Trap: Worst case $O(n)$ with all keys colliding.)
2. [TRAP] Recursion is always less efficient than iteration — True or False? (Trap: Tail recursion is equally efficient; some problems are simpler recursively.)
3. [TRAP] BFS always finds shortest path — True or False? (Trap: Only for unweighted graphs; use Dijkstra for weighted.)
4. [TRAP] Sorting before binary search is always beneficial — True or False? (Trap: If searching once, $O(n)$ linear search $<$ $O(n \log n)$ sort + $O(\log n)$ search.)
5. [TRAP] QuickSort worst case is $O(n \log n)$ — True or False? (Trap: Worst case is $O(n^2)$ on already-sorted arrays with bad pivot.)
6. [TRAP] DFS uses a queue — True or False? (Trap: DFS uses a stack (or recursion); BFS uses a queue.)
7. [TRAP] Hash tables use more memory than arrays — True or False? (Trap: Not always; sparse hash tables can use less memory than dense arrays with wasted slots.)
8. [TRAP] A balanced BST guarantees $O(\log n)$ for all operations — True or False? (Trap: Only if the balancing is maintained; insertion/deletion must rebalance.)
9. [TRAP] Greedy algorithms always give optimal solutions — True or False? (Trap: Only for specific problems with the greedy choice property; e.g., greedy fails for 0/1 knapsack.)
10. [TRAP] DP always uses more memory than greedy — True or False? (Trap: DP can be space-optimized; greedy can need auxiliary structures too.)

DOMAIN 8: SYSTEM DESIGN

--- EASY ---

1. What is the difference between horizontal and vertical scaling?
2. What is a load balancer?
3. What is caching? Name 3 caching strategies.
4. What is the difference between SQL and NoSQL?
5. What is a CDN?
6. What is an API Gateway?
7. What is the difference between monolith and microservices?
8. What is a message queue?
9. What is database replication?
10. What is database sharding?
11. What is a stateless vs stateful service?
12. What is REST vs GraphQL vs gRPC?
13. What is a reverse proxy?
14. What is a forward proxy?
15. What is DNS and how does it work?
16. What is TCP vs UDP?
17. What is HTTP vs HTTPS?
18. What is SSL/TLS?
19. What is latency vs throughput?
20. What is availability vs consistency?
21. What is SLA, SLO, SLI?
22. What is ACID in databases?
23. What is BASE in distributed systems?
24. What is idempotency?
25. What is a webhook?
26. What is polling vs long polling vs WebSocket vs SSE?
27. What is an index in a database?
28. What is a primary key vs foreign key?
29. What is connection pooling?
30. What is a distributed system?

--- MEDIUM ---

1. Design a URL shortener. What are the key decisions?
2. Explain the CAP theorem with a real-world example.
3. How do you design a rate limiter at scale?
4. What is consistent hashing and why is it used?
5. What is eventual consistency? When is it acceptable?
6. Design a notification system for 10M users.
7. What is the difference between optimistic and pessimistic locking?
8. How does a B-tree database index work?
9. What is the SAGA pattern for distributed transactions?
10. What is the circuit breaker pattern?
11. What is the bulkhead pattern in microservices?
12. What is the retry pattern with exponential backoff?
13. What is the difference between synchronous and asynchronous microservices communication?
14. What is an event-driven architecture?
15. What is Kafka? What are producers, consumers, and topics?
16. What is the difference between a message queue and an event bus?
17. What is a two-phase commit (2PC)?
18. What are the failure modes of 2PC?
19. What is leader election in distributed systems?

20. How does a read replica work and when do you use one?
21. What is write-ahead logging (WAL)?
22. What is MVCC (Multi-Version Concurrency Control)?
23. Design a caching layer for a high-read system. What eviction strategies would you use?
24. What is cache-aside vs write-through vs write-behind vs read-through?
25. What is a fanout problem and how do you solve it for a newsfeed?
26. What is a hot partition and how do you handle it?
27. How do you design API versioning?
28. What is service discovery?
29. What is a service mesh?
30. How do you implement distributed tracing?

--- HARD ---

1. Design Twitter for 100M DAU — focus on tweets, timeline, trending.
2. Design YouTube — encoding, storage, streaming, CDN.
3. Design WhatsApp — messaging, delivery guarantees, encryption.
4. Design Uber — matching, location, surge pricing, ETA.
5. Design Google Search — crawling, indexing, ranking.
6. Design a distributed cache like Redis Cluster.
7. Design a real-time collaborative editor (Google Docs).
8. Design a distributed job scheduler with at-least-once guarantee.
9. Design a payment system — what guarantees does it need?
10. Design a stock exchange matching engine.
11. How does Kafka guarantee message ordering and delivery?
12. How does Raft consensus algorithm work?
13. How does Paxos differ from Raft?
14. What is the write path and read path in Cassandra?
15. How does DynamoDB handle partitioning and consistency?
16. How does Google Spanner achieve external consistency?
17. What is a delta lake architecture?
18. What is the lambda architecture for data processing?
19. Design a global CDN — how do you handle cache invalidation at scale?
20. Design a distributed file system (like HDFS).
21. How does consistent hashing with virtual nodes solve hot spots?
22. Design a real-time leaderboard system for 10M players.
23. Design a multi-region database with low-latency reads globally.
24. Design a fraud detection system that must respond in < 100ms.
25. How would you design a recommendation system for Netflix?
26. Design a chat system that scales to 1B messages/day.
27. How do you implement backpressure in a microservices system?
28. How do you design a system that guarantees exactly-once delivery?
29. Design an API rate limiter that works across multiple data centers.
30. Design a distributed lock service.

--- SUPER HARD ---

1. Design the infrastructure for a global social network with 5B users — every layer.
2. How would you design a database engine from scratch?
3. Design a globally distributed strongly consistent key-value store.
4. How would you design the infrastructure for autonomous vehicle fleet management in real time?
5. Design a system that ingests 1 petabyte of data per day with sub-second query latency.
6. How would you design a chaos engineering platform like Netflix's Chaos Monkey?
7. Design a multi-cloud deployment architecture with failover between AWS, GCP, and Azure.
8. How would you design a zero-trust security architecture for a microservices system?
9. Design a system for real-time ML inference serving 1M predictions/second.

10. How would you design a blockchain-based voting system that is secure, private, and auditable?

--- TRAP QUESTIONS ---

1. [TRAP] Microservices are always better than monoliths — True or False? (Trap: Microservices add operational complexity; monoliths often win for early-stage products.)
2. [TRAP] Adding a cache always improves performance — True or False? (Trap: Cache invalidation bugs serve stale data; cold start can make things worse.)
3. [TRAP] Database replication gives both high availability AND zero data loss — True or False? (Trap: Async replication can lose recent writes during failover.)
4. [TRAP] Horizontal scaling solves every bottleneck — True or False? (Trap: Stateful services and shared locks require rearchitecting before scaling horizontally.)
5. [TRAP] Kafka guarantees exactly-once delivery out of the box — True or False? (Trap: Exactly-once requires idempotent producers AND transactional APIs, both correctly configured.)
6. [TRAP] SQL databases cannot scale horizontally — True or False? (Trap: Postgres can scale read replicas; Vitess shards MySQL horizontally.)
7. [TRAP] CDN always reduces latency — True or False? (Trap: For dynamic, personalized content, CDN adds cache miss overhead.)
8. [TRAP] Microservices always communicate faster than monolith internal calls — True or False? (Trap: Network calls between microservices are orders of magnitude slower than in-process function calls.)
9. [TRAP] Eventual consistency means data can be permanently wrong — True or False? (Trap: Eventual consistency guarantees convergence; it just doesn't specify when.)
10. [TRAP] A single Kafka partition can handle unlimited throughput — True or False? (Trap: Partitions are bounded by broker disk I/O and network bandwidth.)

DOMAIN 9: JAVA DEVELOPER

--- EASY ---

1. What is the difference between JDK, JRE, and JVM?
2. What are the 4 pillars of OOP in Java?
3. What is the difference between `==` and `.equals()`?
4. What is the difference between an interface and an abstract class?
5. What is autoboxing and unboxing?
6. What is the difference between `ArrayList` and `LinkedList`?
7. What is the difference between `HashMap` and `TreeMap`?
8. What is a checked vs unchecked exception?
9. What is the `final` keyword in Java?
10. What is the difference between `String`, `StringBuilder`, and `StringBuffer`?
11. What is the difference between method overloading and overriding?
12. What is a constructor? Can it be private?
13. What is the `static` keyword?
14. What is the `this` keyword?
15. What is the `super` keyword?
16. What are generics in Java?
17. What is a wildcard in generics?
18. What is the difference between `Iterator` and `for-each` loop?
19. What is Java's garbage collector?
20. What is the difference between `int` and `Integer`?
21. What is a `varargs` in Java?
22. What is a nested class vs inner class?
23. What is an anonymous class?
24. What is the `instanceof` operator?
25. What is the `transient` keyword?
26. What is the `volatile` keyword?
27. What is serialization in Java?
28. What is the difference between `throw` and `throws`?
29. What are Java 8 features? Name 5.
30. What is the `Optional` class in Java?

--- MEDIUM ---

1. Explain the Java memory model — heap, stack, metaspace, code cache.
2. What is the difference between `Comparable` and `Comparator`?
3. Explain the Java Collections Framework hierarchy.
4. What are functional interfaces? Name 5 built-in ones.
5. What is the difference between `Predicate`, `Function`, `Consumer`, and `Supplier`?
6. Explain the Stream API with a complex pipeline example.
7. What is the difference between `map()`, `flatMap()`, and `filter()` in streams?
8. What is the difference between `HashMap` and `ConcurrentHashMap`?
9. What is a fail-fast vs fail-safe iterator?
10. How does `HashMap` work internally in Java?
11. What is the load factor and rehashing in `HashMap`?
12. What is Java's memory model (JMM) and happens-before relationship?
13. What is the `synchronized` keyword? What are its limitations?
14. What is `ReentrantLock`? How does it differ from `synchronized`?
15. What is the difference between `wait/notify` and `Condition` in Java?
16. What is a `ThreadLocal` variable?
17. What is the `ExecutorService` framework?
18. What is `CompletableFuture` in Java 8+?
19. What is the difference between `Future` and `CompletableFuture`?

20. What is Spring Boot and how does auto-configuration work?
21. What is dependency injection and IoC?
22. What is Spring Bean lifecycle?
23. What are Spring Bean scopes?
24. What is @Transactional in Spring? What are the propagation levels?
25. What is Spring Data JPA?
26. What is JPQL vs native query?
27. What is the N+1 query problem in JPA?
28. What are Spring Boot Actuator endpoints?
29. What is the difference between @Controller and @RestController?
30. What is Spring Security? How does filter chain work?

--- HARD ---

1. How does the JVM garbage collector work? Compare G1GC, ZGC, and Shenandoah.
2. What is JIT compilation? What optimizations does JVM perform?
3. How does JVM class loading work? What is the class loader hierarchy?
4. What is Java bytecode? How does the JVM interpret and JIT it?
5. Explain the ForkJoin framework and work-stealing algorithm.
6. Implement a thread-safe producer-consumer using BlockingQueue.
7. How do you detect and fix a deadlock in Java?
8. What is lock striping and how is it used in ConcurrentHashMap?
9. What are Java agents and how does instrumentation work?
10. What is Project Loom (virtual threads)?
11. How do virtual threads differ from platform threads?
12. What is record types in Java 16+?
13. What is sealed classes in Java 17+?
14. What is pattern matching in Java?
15. Implement a thread pool executor from scratch in Java.
16. How do you implement a distributed lock using Zookeeper from Java?
17. What is reactive programming? How does Project Reactor / RxJava work?
18. What is the difference between Mono and Flux in Project Reactor?
19. What is backpressure in reactive streams?
20. How does Spring WebFlux differ from Spring MVC?
21. How do you profile a Java application for memory leaks?
22. What is the Java Flight Recorder (JFR)?
23. How would you optimize a Java application for low-latency trading?
24. What is off-heap memory in Java and when would you use it?
25. Implement a concurrent LRU cache in Java.
26. How does Spring AOP work internally?
27. What is CGLIB proxy vs JDK dynamic proxy in Spring?
28. How do you implement a custom Spring Boot starter?
29. What is the event sourcing pattern in Java Spring?
30. How would you implement a CQRS architecture in Spring Boot?

--- SUPER HARD ---

1. Design a low-latency trading system in Java — what JVM tuning and data structures would you use?
2. How would you implement a distributed transaction across multiple Spring Boot microservices using SAGA?
3. Implement a lock-free concurrent queue in Java using CAS operations.
4. How would you implement a custom ClassLoader in Java and what are the use cases?
5. Design a Java-based stream processing engine that handles 1M events/second.
6. How would you implement Java serialization that is backward and forward compatible?
7. Build a Java framework for building REST APIs with automatic OpenAPI documentation.
8. How does the G1GC pause-time target work internally? How would you tune it for 99.9th percentile SLA?
9. Implement a distributed cache in Java with replication, eviction, and consistency guarantees.

10. Design a Java-based plugin system where plugins are loaded, isolated, and updated at runtime.

--- TRAP QUESTIONS ---

1. [TRAP] Java is always pass-by-value — True or False? (Trap: True, but for objects the value IS the reference — so the object's fields can be mutated.)
2. [TRAP] String is immutable so it's always thread-safe — True or False? (Trap: The String object is safe, but the reference to it is not if shared without synchronization.)
3. [TRAP] Static methods can be overridden — True or False? (Trap: They can be hidden/shadowed but not overridden — no polymorphism.)
4. [TRAP] int and Integer are interchangeable — True or False? (Trap: Integer can be null, auto-unboxing null Integer throws NullPointerException.)
5. [TRAP] synchronized on a method locks the entire class — True or False? (Trap: It locks the INSTANCE (this), not the class. Use static synchronized for class-level locking.)
6. [TRAP] Making a field volatile guarantees atomicity — True or False? (Trap: Volatile guarantees visibility but NOT atomicity. i++ is not atomic even on volatile.)
7. [TRAP] Java Strings are stored in the heap — True or False? (Trap: String literals are stored in the String Pool in the heap; but the object reference is on the stack.)
8. [TRAP] Collections.synchronizedList is always safe for concurrent iteration — True or False? (Trap: You must manually synchronize during iteration with synchronized(list){...}.)
9. [TRAP] @Transactional works on private methods in Spring — True or False? (Trap: Spring AOP proxies can't intercept private methods; @Transactional is ignored.)
10. [TRAP] HashMap is ordered — True or False? (Trap: HashMap is unordered; use LinkedHashMap for insertion order, TreeMap for sorted order.)

SECTION B: BEHAVIOURAL QUESTIONS (STAR Method)

Answer all behavioural questions using the STAR method: Situation, Task, Action, Result.

--- FOR FRESHERS ---

1. Tell me about yourself. (Structure: background, skills, projects, why this role.)
2. Tell me about a project you are most proud of.
3. Describe a time you worked under pressure to meet a deadline.
4. Tell me about a time you learned a new technology or skill quickly.
5. Describe a time you made a mistake. How did you handle it?
6. Have you ever had a conflict with a teammate? How did you resolve it?
7. Tell me about a time you took initiative without being asked.
8. Describe a situation where you received critical feedback. How did you respond?
9. Tell me about a time you failed. What did you learn from it?
10. Describe a time you had to manage multiple tasks simultaneously.
11. Tell me about a time you went above and beyond for a project.
12. Describe a time you had to explain a complex concept to a non-technical person.
13. Tell me about a time you disagreed with your professor or mentor. What happened?
14. Describe your most challenging academic or project experience.
15. Tell me about a time you had to work with someone who had a completely different working style.
16. Describe a time when your plan didn't work out. What did you do?
17. Tell me about a time you had to self-study to complete a task.
18. Describe a situation where you showed leadership in a group project.
19. Tell me about a time you improved a process or approach.
20. Describe a time you had to prioritize between multiple competing tasks.

--- FOR EXPERIENCED CANDIDATES ---

1. Tell me about the most complex system you have built. What made it complex?
2. Describe a time you led a team through a major technical challenge.
3. Tell me about a time you made a critical architectural decision that turned out to be wrong.
4. Describe a time you had to convince stakeholders to adopt a new technology.
5. Tell me about a production outage you handled. What was your approach?
6. Describe a time you mentored a junior developer. What was your approach?
7. Tell me about a time you pushed back on a product requirement. What happened?
8. Describe a situation where your team was missing a deadline. What did you do?
9. Tell me about a time you had to refactor a large, messy codebase. How did you approach it?
10. Describe a situation where you had to onboard a new team member quickly.
11. Tell me about a time you identified a security vulnerability in production.
12. Describe a time you had to make a critical decision with incomplete information.
13. Tell me about a time you had to balance technical debt with feature delivery.
14. Describe a situation where two senior engineers disagreed. How did you navigate it?
15. Tell me about a time you improved team processes or engineering culture.
16. Describe a time you had to deliver a feature you personally disagreed with.
17. Tell me about the most difficult debugging experience you've had.
18. Describe a time you failed to meet a commitment. How did you handle it?
19. Tell me about a time you had to scale a system under extreme time pressure.
20. Describe a situation where you had to manage conflict between product and engineering.

SECTION C: HR QUESTIONS

--- STANDARD HR ---

1. Tell me about yourself.
2. Why do you want to work at our company?
3. Why are you leaving your current job? (For experienced)
4. Where do you see yourself in 5 years?
5. What is your greatest strength?
6. What is your greatest weakness?
7. Why should we hire you?
8. What are your salary expectations?
9. Are you comfortable working in a remote / hybrid / on-site environment?
10. How do you handle stress and pressure?
11. Are you a team player or do you prefer working independently?
12. What do you know about our company and our products?
13. What motivates you?
14. Do you have any questions for us?
15. How soon can you join?
16. Are you open to relocation?
17. Are you interviewing anywhere else?
18. What is your biggest career achievement so far?
19. How would your previous teammates describe you?
20. What is your management style? (For senior roles)
21. How do you stay updated with the latest technology trends?
22. What does success look like to you in this role?
23. How do you handle feedback?
24. What type of work environment do you thrive in?
25. What are your long-term career goals?
26. Do you prefer working on frontend, backend, or full-stack?
27. Have you worked in an agile/scrum environment?
28. How do you prioritize when everything is urgent?
29. What would you do in your first 90 days?
30. How do you handle working on multiple projects simultaneously?

--- TRAP HR QUESTIONS ---

These questions are designed to reveal character, self-awareness, and potential red flags.

1. [TRAP] If your previous manager was watching this interview, what would they say about you? (Tests self-awareness AND honesty.)
2. [TRAP] What was the most frustrating thing about your last job? (Trap: Venting about old employer is a red flag.)
3. [TRAP] If you got this job, how long would you stay? (Trap: 'Forever' sounds fake; 'short-term' raises concerns.)
4. [TRAP] What would make you leave this company after joining? (Tests cultural fit and loyalty signals.)
5. [TRAP] How do you handle a task given to you that is outside your job description? (Tests flexibility vs entitlement.)
6. [TRAP] Describe your ideal manager. (Trap: May reveal incompatibility with the actual manager.)
7. [TRAP] What do you think is wrong with our product/company? (Tests research depth, constructive criticism ability.)
8. [TRAP] If given two competing priorities with the same deadline, what would you do? (Tests decision-making and communication.)
9. [TRAP] How honest are you? On a scale of 1-10? (Trap: Saying 10 sounds arrogant; low sounds bad. Best answer: 8 with explanation.)
10. [TRAP] What is something you cannot do that we might expect you to do? (Tests self-awareness and honesty.)
11. [TRAP] If you could change one thing about your last team, what would it be? (Trap: Criticizing teams is risky.)
12. [TRAP] Are you overqualified for this position? (Trap: Saying no when you are is unconvincing; need a genuine answer.)
13. [TRAP] What would you do if you discovered your manager was doing something unethical? (Tests integrity vs loyalty.)
14. [TRAP] If we don't offer you this job, what will you do? (Tests desperation level vs confidence.)

15. [TRAP] How would you rate yourself as a developer on a scale of 1-10? (Trap: 10 is arrogant; 5 is too low. Best: 7 with specifics.)

SECTION D: SITUATIONAL QUESTIONS

--- FRESHER SCENARIOS ---

1. You are given a task you have never done before and your senior is on leave. What do you do?
2. You discover a bug in production 30 minutes before an important client demo. What are your steps?
3. A teammate consistently misses their deadlines in a group project. How do you handle it?
4. Your manager asks you to implement a feature in a technology you have never used. How do you respond?
5. You finish your assigned task two days early. What do you do with the remaining time?
6. You are asked to estimate how long a feature will take, but you genuinely have no idea. What do you say?
7. Two senior developers are arguing about architecture and ask your opinion. As a fresher, what do you do?
8. You realize mid-sprint that your technical approach is completely wrong. What do you do?
9. You disagree with a code review comment from your senior. How do you handle it?
10. You accidentally push broken code to the main branch. What do you do?
11. Your manager sets a deadline that you believe is impossible. What do you say?
12. A client is asking for a feature that is technically impossible as described. How do you respond?
13. You are working on a feature and realize it will require changing something another team owns. What do you do?
14. Your internet connection fails during an important remote meeting. How do you handle it?
15. You are onboarded to a project with no documentation. How do you get up to speed?
16. You are asked to review a senior developer's code and find a bug. How do you handle the feedback?
17. Two teammates have a personal conflict that is affecting team performance. What do you do?
18. You are given access to production credentials for a task. What precautions do you take?
19. A feature you built went live and is causing performance issues. What do you do?
20. You have a question about a task but your manager seems very busy. How do you handle it?

--- EXPERIENCED SCENARIOS ---

1. Your team is pressured to ship a feature with known security vulnerabilities to meet a deadline. What do you do?
2. You are onboarded to a 5-year-old monolith with no documentation and no tests. How do you approach it?
3. A client reports the system is down. You have no monitoring and no logs. Walk through your incident response.
4. Your team wants to rewrite the entire backend in a new language. How do you evaluate this decision?
5. A junior developer's PR has serious design flaws but they are emotionally invested. How do you handle the code review?
6. You discover that a key design decision you approved 6 months ago is causing major scaling issues. What do you do?
7. Your manager asks you to implement something technically impossible in the given timeline. How do you respond?
8. Two teams are in conflict over API contract ownership. You are the tech lead. How do you resolve it?
9. A competitor releases a feature that makes your current roadmap obsolete. How do you handle it as an engineer?
10. You are given the task to reduce infrastructure costs by 40% without reducing performance. How do you approach it?
11. A critical third-party API your system depends on goes down. What is your incident management approach?
12. You find a major performance bottleneck that will take 3 weeks to fix but the team is focused on new features. What do you do?
13. A senior developer on your team is resistant to adopting code reviews. How do you handle it?
14. You are asked to hire a developer for your team. What does your interview process look like?
15. Your company decides to migrate from AWS to GCP. You are the lead engineer. What is your plan?
16. You discover that a colleague has committed sensitive API keys to a public GitHub repository. What do you do immediately?
17. You are leading a project and realize 2 weeks before launch that you will miss the deadline. What do you do?
18. Your team's velocity has dropped by 50% over the last quarter. How do you investigate and address it?
19. A stakeholder bypasses you and directly assigns tasks to your team members. How do you handle it?
20. You are asked to estimate the cost of building a new platform from scratch. How do you approach the estimate?

--- EDGE CASE AND ETHICAL SCENARIOS ---

1. You discover that a feature you built is being used to collect user data in a way users didn't consent to. What do you do?
2. A manager asks you to backdate a commit to make it look like a feature was delivered on time. What do you do?
3. You find out your company is violating GDPR regulations in how it stores user data. What are your steps?
4. A colleague asks you to approve their PR without reviewing it because they are in a hurry. What do you say?

5. You are asked to reverse-engineer a competitor's software. What do you do?
6. Your company asks you to build a feature that you personally believe is harmful to users. How do you respond?
7. You make a mistake that costs the company money. How do you handle the aftermath?
8. A recruiter from a competitor contacts you on LinkedIn with an offer. How do you handle it professionally?
9. You suspect a colleague is using company resources for personal projects. What do you do?
10. You are the only one who knows how a critical system works because the original developer left. How do you handle this responsibility?

END OF QUESTION BANK | AI Interview Platform | Total: 800+ Questions across 9 domains + Behavioural + HR + Situational